CHAPTER FOUR

SOFTWARE DESIGN

4.0 Introduction

Chapter Three analysed the requirements of the AI Agent for Real Estate Recommendation and Price Prediction System and modelled its processes and data. This chapter translates that analysis into a design that can be implemented. It presents the architectural design of the system, the design of the user interface, the way in which data are stored, the design of the database, including normalization, the transformation of the entity relationship diagram into relations and the data dictionaries, and the design of the forms and reports of the system.

4.1 Architectural Design

The system follows a three-layer web architecture, shown in Figure 4.1. The presentation layer is the web browser, through which the Public Visitor, the Customer, the Property Manager and the Administrator reach the public portal and the three role-based areas. The application layer is a single Next.js application written in TypeScript. It contains the React user interface, a middleware component that authenticates users and restricts each area to its role, the Route Handlers that provide the backend services, and the three AI modules identified in Chapter Three. The data layer consists of the Prisma ORM, the MySQL database and the file storage that holds property images.

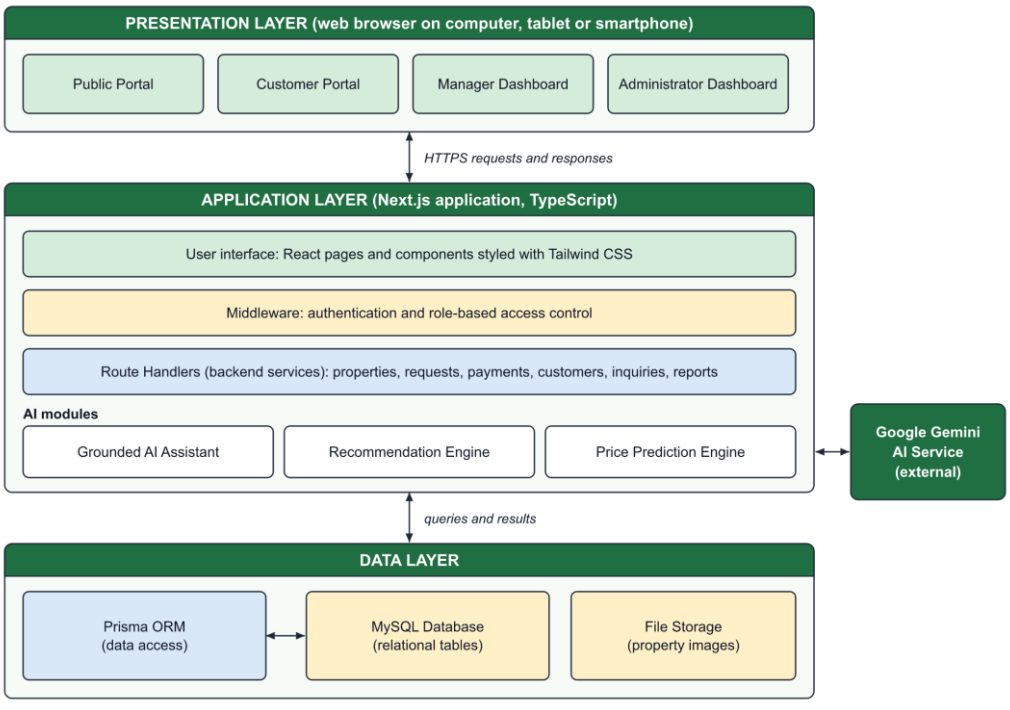


Figure 4.1: Architectural Design of the Proposed System

This architecture was chosen for three reasons. First, the interface and the backend services share one codebase and one language, which reduces the effort of development and maintenance. Second, every request passes through the middleware and the Route Handlers before it reaches the database, so the business rules of Chapter Three, such as the rule that only the Property Manager approves rental bookings, are enforced in one place. Third, the AI modules are separated from the business logic: the recommendation and price prediction engines run inside the application, and only the AI assistant communicates with an external service, Google Gemini, to which it sends the question together with the relevant property records.

4.2 User Interface Design

The user interface is designed around the four actors of the system. Figure 4.2 shows the navigation structure. All users enter through the home page. Public pages are available without an account, and after login the system redirects each user to the area of the user's role: the customer portal, the manager dashboard or the administrator dashboard. A user cannot open an area that belongs to another role.

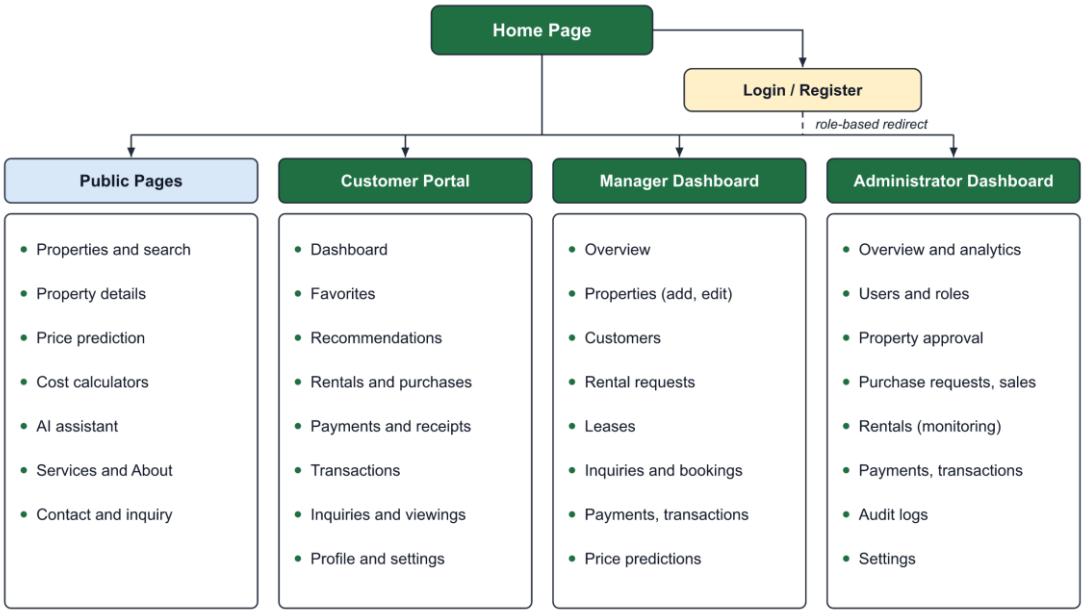


Figure 4.2: Navigation Structure of the User Interface

The interface follows four design principles. Consistency: every page uses the same header, colours, typography and button styles, and the three dashboards share one layout of a sidebar, summary cards and data tables. Simplicity: a visitor can search for a property from the first screen without registering. Responsiveness: the layout adapts to the width of the screen, so tables become cards on small screens. Feedback: the status of every request and payment is

displayed with a labelled badge. The interface is presented under the name Kiro-Maal Real Estate. Figure 4.3 shows the implemented home page and the property search interface.

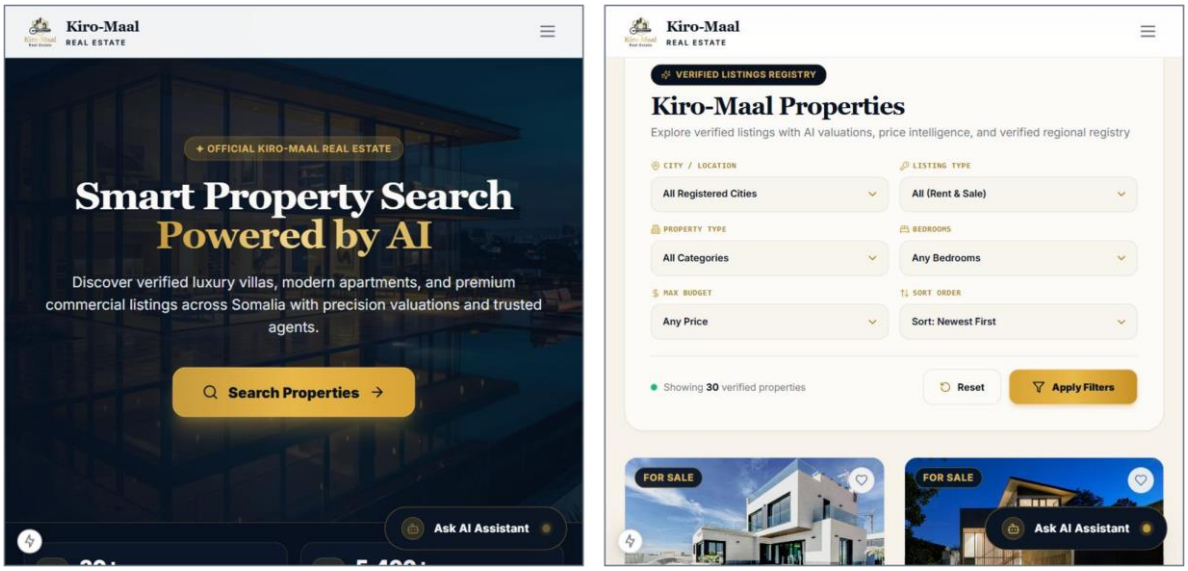


Figure 4.3: Home Page and Property Search Interface

4.3 Database Storage Design

The data of the system are stored in a MySQL relational database. Each entity of the ERD in Chapter Three is stored as one table, each record as one row, and each relationship as a foreign key. The tables are created and changed through the Prisma schema, which keeps the structure of the database and the application code consistent. Property images are not stored inside the database; the image files are kept in the file storage of the application, and only their addresses are stored in the PROPERTY_IMAGES table, which keeps the database small and queries fast. Passwords are never stored in readable form; only their hashes are stored. Table 4.1 lists the data types used.

Table 4.1: Data Types Used in the Database

Data Type	Use in the Proposed System
VARCHAR	Identifiers, names, e-mail addresses, cities, request and receipt numbers
TEXT	Long text such as property descriptions, messages and review notes
INT	Whole numbers such as bedrooms, bathrooms and view counts
DOUBLE	Prices, rent, deposits, areas and geographic coordinates
BOOLEAN	Yes or no values such as is_active, is_furnished and is_read
DATETIME	Dates and times such as created_at, start_date, end_date and paid_at
ENUM	Fixed lists of values such as role, property type and status

4.4 Database Design

The database design converts the conceptual model of Chapter Three into tables that are free from redundancy. This section explains normalization, transforms the ERD into relations and defines the data dictionaries.

4.4.1 Database Normalization

Normalization is the process of organizing data into tables so that each fact is stored once and update anomalies are avoided (Elmasri & Navathe, 2016). It is illustrated below with the rental data of the system, beginning with an unnormalized record in which all the details of a rental are written together: RENTAL (request_no, customer_name, customer_phone, property_title, property_city, manager_name, start_date, end_date, rent_amount, payment_1, payment_2, ...).

4.4.1.1 First Normal Form (1NF)

A table is in first normal form when every field holds a single value and there are no repeating groups. In the unnormalized record the payments form a repeating group. They are therefore moved to a separate table in which each payment is one row: RENTAL (request_no, customer_name, customer_phone, property_title, property_city, manager_name, start_date, end_date, rent_amount) and PAYMENT (payment_id, request_no, amount, payment_method, paid_at).

4.4.1.2 Second Normal Form (2NF)

A table is in second normal form when it is in 1NF and every non-key field depends on the whole primary key. Each table of the system has a single-field primary key, such as rental_request_id, so no field can depend on only part of a key. Junction data are treated in the same way: FAVORITES has its own key, favorite_id, and stores only the two foreign keys and the date.

4.4.1.3 Third Normal Form (3NF)

A table is in third normal form when it is in 2NF and no non-key field depends on another non-key field. In the rental record, customer_phone depends on the customer and property_city depends on the property, not on the rental. These fields are therefore removed to the tables to which they belong, and the rental keeps only foreign keys: USERS (user_id, name, phone, ...), PROPERTIES (property_id, title, city, ...) and RENTAL_REQUESTS (rental_request_id, request_no, customer_id, property_id, manager_id, start_date, end_date, rent_amount, status).

4.4.1.4 Normalized Tables of the Proposed System

Applying the same steps to all the data of the system produces the normalized tables listed in Table 4.2.

Table 4.2: Normalized Tables of the Proposed System

Table	Primary Key	Foreign Keys
users	user_id	None
properties	property_id	manager_id
property_images	image_id	property_id
favorites	favorite_id	user_id, property_id
search_history	search_id	user_id
recommendations	recommendation_id	user_id, property_id
price_predictions	prediction_id	user_id, property_id
rental_requests	rental_request_id	customer_id, property_id, manager_id
purchase_requests	purchase_request_id	customer_id, property_id, reviewed_by
transactions	transaction_id	customer_id, property_id, rental_request_id, purchase_request_id
payments	payment_id	transaction_id, customer_id
receipts	receipt_id	payment_id, transaction_id
inquiries	inquiry_id	property_id, customer_id, manager_id
appointments	appointment_id	property_id, customer_id, manager_id
reviews	review_id	property_id, user_id
notifications	notification_id	user_id
audit_logs	log_id	user_id

4.4.2 Transforming E-R Diagrams into Relations

The ERD of Chapter Three is transformed into relations by three rules. Each entity becomes a relation whose primary key is the identifier of the entity. Each one-to-many relationship is represented by placing the primary key of the "one" side as a foreign key in the relation of the "many" side. Each many-to-many relationship becomes a relation of its own that contains the keys of both sides. Applying these rules gives the following main relations, in which the primary key is written first and foreign keys are marked (FK).

USERS (user_id, name, email, phone, password_hash, role, is_active, created_at)

PROPERTIES (property_id, manager_id (FK), title, description, type, listing_type, status, price, city, location, bedrooms, bathrooms, area, amenities)

FAVORITES (favorite_id, user_id (FK), property_id (FK), created_at)

RENTAL_REQUESTS (rental_request_id, request_no, customer_id (FK), property_id (FK), manager_id (FK), start_date, end_date, rent_amount, security_deposit, status)

PURCHASE_REQUESTS (purchase_request_id, request_no, customer_id (FK), property_id (FK), reviewed_by (FK), sale_price, status, reviewed_at, completed_at)

TRANSACTIONS (transaction_id, txn_no, type, customer_id (FK), property_id (FK), rental_request_id (FK), purchase_request_id (FK), amount, status)

PAYMENTS (payment_id, transaction_id (FK), customer_id (FK), amount, payment_method, status, transaction_ref, paid_at)

RECEIPTS (receipt_id, receipt_no, payment_id (FK), transaction_id (FK), issued_at)

4.4.3 Data Dictionaries

A data dictionary describes each field of a table by its data type, its constraint and its meaning. Tables 4.3 to 4.8 present the data dictionaries of the six central tables of the system. The field names follow the ERD of Chapter Three; identifiers are stored as text keys generated by the system.

Table 4.3: Data Dictionary of the Users Table

Field	Data Type	Constraint	Description
user_id	VARCHAR	Primary key	Unique identifier of the user
name	VARCHAR	Not null	Full name
email	VARCHAR	Unique, not null	E-mail address used to log in
phone	VARCHAR	Optional	Telephone number
password_hash	VARCHAR	Not null	Hashed password
role	ENUM	Not null	Administrator, Property Manager or Customer
is_active	BOOLEAN	Default true	Active or suspended account
created_at	DATETIME	Not null	Date of registration

Table 4.4: Data Dictionary of the Properties Table

Field	Data Type	Constraint	Description
property_id	VARCHAR	Primary key	Unique identifier of the property
manager_id	VARCHAR	Foreign key	Property Manager assigned to the property
title	VARCHAR	Not null	Title of the listing
description	TEXT	Not null	Description of the property
type	ENUM	Not null	House, apartment, villa, office, land and others
listing_type	VARCHAR	Not null	For sale or for rent
status	ENUM	Default pending	Draft, pending, approved, published, rented, sold, archived
price	DOUBLE	Not null	Sale price or rent
city, location	VARCHAR	Not null	City and district of the property

Field	Data Type	Constraint	Description
bedrooms, bathrooms	INT	Not null	Number of bedrooms and bathrooms
area	DOUBLE	Not null	Area in square metres
amenities	TEXT	Optional	List of amenities

Table 4.5: Data Dictionary of the Rental Requests Table

Field	Data Type	Constraint	Description
rental_request_id	VARCHAR	Primary key	Unique identifier of the rental request
request_no	VARCHAR	Unique	Reference number shown to users
customer_id	VARCHAR	Foreign key	Customer who rents
property_id	VARCHAR	Foreign key	Property that is rented
manager_id	VARCHAR	Foreign key	Property Manager who decides on the request
start_date, end_date	DATETIME	Not null	Rental period
rent_amount	DOUBLE	Not null	Rent for one period
security_deposit	DOUBLE	Not null	Deposit paid by the customer
status	ENUM	Default pending	Pending, approved, rejected, active, expired, cancelled

Table 4.6: Data Dictionary of the Purchase Requests Table

Field	Data Type	Constraint	Description
purchase_request_id	VARCHAR	Primary key	Unique identifier of the purchase request
request_no	VARCHAR	Unique	Reference number shown to users
customer_id	VARCHAR	Foreign key	Customer who buys
property_id	VARCHAR	Foreign key	Property that is bought
sale_price	DOUBLE	Not null	Agreed sale price
status	ENUM	Default pending	Pending, payment pending, approved, completed, rejected
reviewed_by	VARCHAR	Foreign key	Administrator who reviewed the request
reviewed_at, completed_at	DATETIME	Optional	Dates of review and completion

Table 4.7: Data Dictionary of the Transactions Table

Field	Data Type	Constraint	Description
transaction_id	VARCHAR	Primary key	Unique identifier of the transaction
txn_no	VARCHAR	Unique	Transaction number
type	ENUM	Not null	Sale or rental
customer_id	VARCHAR	Foreign key	Customer of the transaction
property_id	VARCHAR	Foreign key	Property of the transaction
rental_request_id	VARCHAR	Foreign key, unique	Related rental request, if any
purchase_request_id	VARCHAR	Foreign key, unique	Related purchase request, if any

Field	Data Type	Constraint	Description
amount	DOUBLE	Not null	Amount of the transaction
status	ENUM	Default pending	Pending, paid, completed, active, rejected, cancelled

Table 4.8: Data Dictionary of the Payments Table

Field	Data Type	Constraint	Description
payment_id	VARCHAR	Primary key	Unique identifier of the payment
transaction_id	VARCHAR	Foreign key	Transaction settled by the payment
customer_id	VARCHAR	Foreign key	Customer who paid
amount	DOUBLE	Not null	Amount paid
payment_method	VARCHAR	Not null	Card, bank wire, cash or mobile money
status	ENUM	Not null	Pending, paid, failed or refunded
transaction_ref	VARCHAR	Unique	Payment reference
paid_at	DATETIME	Optional	Date and time of payment

4.5 Designing Forms and Reports

Forms are the means by which users enter data, and reports are the means by which the system presents stored data. The forms of the system follow common rules: related fields are grouped, required fields are marked, lists are used wherever the possible values are fixed, such as city and property type, and each form is validated before it is saved, with the error shown beside the field. The main forms are the login and registration form, the multi-step property form of the Property Manager, the rental booking form, the purchase request form, the payment form, the inquiry form and the price prediction form. Figure 4.4 shows two of the implemented forms.

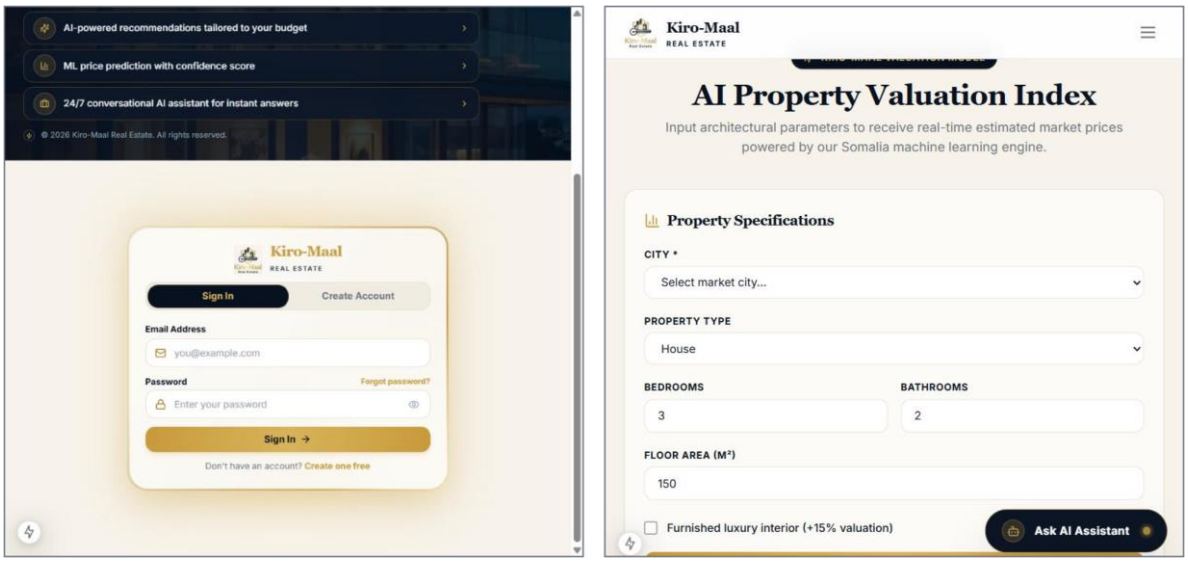


Figure 4.4: Login Form and Price Prediction Form

The reports of the system are the role dashboards, the lists of properties, customers, rentals and requests, the central transaction ledger, the analytics of the Administrator and the digital receipt. Lists are presented as tables that can be searched, filtered and sorted, and a status is always shown as a labelled badge. The receipt is designed as a printable document that can be exported as PDF. Figure 4.5 shows the layout designed for the dashboards and for the receipt.

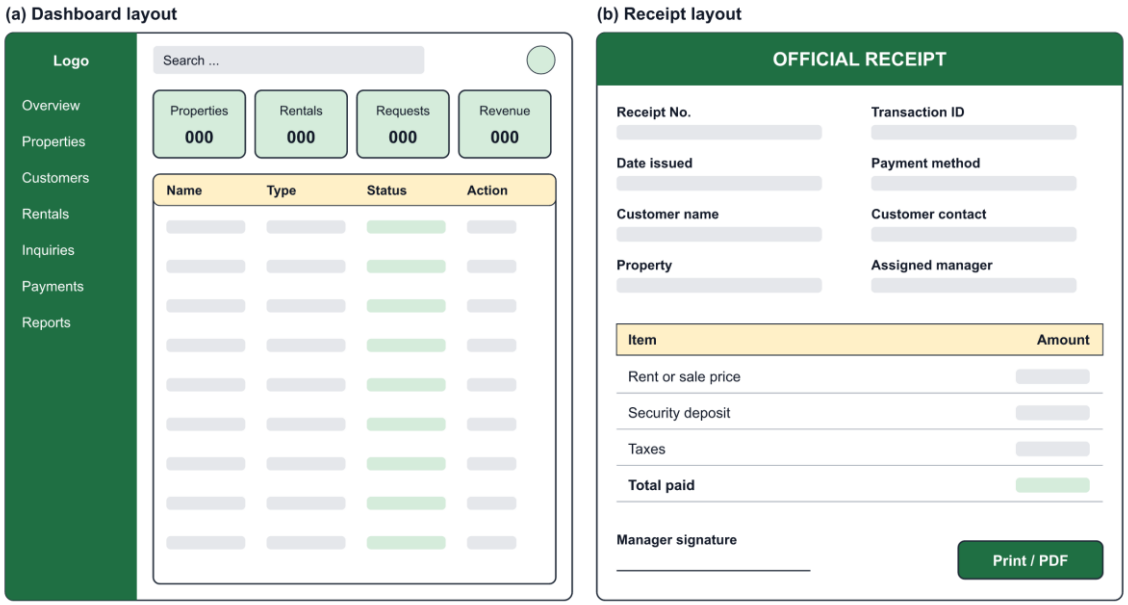


Figure 4.5: Layout Design of the Dashboards and the Receipt

4.6 Chapter Summary

This chapter presented the design of the system. The architectural design organized the system into presentation, application and data layers, and the user interface design defined the navigation of the four actors and the principles of the interface. The database storage design and the database design described how data are stored in MySQL, normalized the data to third normal form, transformed the ERD into relations and defined the data dictionaries of the central tables. Finally, the forms and reports of the system were designed. The next chapter describes how this design was implemented and tested.

CHAPTER FIVE

SYSTEM IMPLEMENTATION

5.0 Introduction

This chapter describes how the design presented in Chapter Four was turned into a working system. It explains the coding phase, including the tools, the structure of the code and examples of the implementation, then the testing of the system at the development, release and user levels, and finally the user manual for each actor.

5.1 Coding Phase

The system was coded as a single Next.js application in TypeScript, using the technologies selected in Chapter Three. Table 5.1 lists the tools used in the coding phase.

Table 5.1: Implementation Tools

Tool	Use in the Coding Phase
Next.js and React	Pages, components and server-side functions of the application
TypeScript	Programming language of the interface, backend services and AI modules
Tailwind CSS	Styling and responsive layout
Prisma ORM	Definition of the database schema and typed access to the data
MySQL	Storage of the data of the system
NextAuth and bcryptjs	Session-based authentication and hashing of passwords
Google Gemini AI service	Natural language responses of the AI assistant
Visual Studio Code, Git and GitHub	Code editing and version control

The code is organized by responsibility, as shown in Table 5.2. Each area of the interface has its own folder, the backend services are grouped under one folder, and the AI modules are kept in a separate library so that they can be developed and tested independently.

Table 5.2: Structure of the Source Code

Folder	Content
app/(public)	Public portal: home, properties, price prediction, AI assistant, contact
app/(auth)	Login and registration pages
app/(customer)	Customer portal: favorites, recommendations, rentals, purchases, payments
app/(dashboard)	Manager dashboard: properties, customers, rentals, inquiries, bookings
app/(admin)	Administrator dashboard: users, approvals, sales, transactions, analytics
app/api	Route Handlers: properties, requests, payments, customers, AI services
components	Reusable interface components
lib and lib/ai	Business logic and AI modules: assistant, recommendation, valuation
prisma	Database schema

Database implementation. Each table of the data dictionary was implemented as a model in the Prisma schema, from which the MySQL tables are generated. Listing 5.1 shows the model of the FAVORITES table. The two relation lines implement the foreign keys, and the unique constraint prevents a customer from saving the same property twice.

```
model Favorite {
  id         String   @id @default(cuid())
  userId     String
  propertyId String
  createdAt  DateTime @default(now())

  user     User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  property Property @relation(fields: [propertyId], references: [id], onDelete: Cascade)

  @@unique([userId, propertyId])
  @@map("favorites")
}
```

Listing 5.1: Prisma Model of the Favorites Table

Role-based access. The role boundaries defined in Chapter Three are enforced by a middleware that runs before every protected page. Listing 5.2 shows the part that protects the administrator area: a visitor who is not logged in is sent to the login page, and a logged-in user with another role is returned to the user's own area.

```
if (isAdminRoute) {
  if (!isLoggedIn) {
    return NextResponse.redirect(new URL("/login", nextUrl));
  }
  if (role !== "ADMIN") {
    const destination = role === "CUSTOMER" ? "/customer" : "/dashboard";
    return NextResponse.redirect(new URL(destination, nextUrl));
  }
  return NextResponse.next();
}
```

Listing 5.2: Middleware Protecting the Administrator Area

AI modules. The recommendation engine scores every available property against the preferences of the Customer. Listing 5.3 shows the weights of the content-based score. The score is then adjusted with preferences learned from the searches, favorites and views of the Customer, and each recommendation is saved with its reasons. The price prediction engine prepares the features of a property, namely city, property type, area, bedrooms, bathrooms and related attributes, and applies a trained regression model to return an estimate with a range. Ridge regression, Random Forest and Gradient Boosting models are implemented in the valuation module. The AI assistant retrieves the property records relevant to a question from the database and sends them with the question to the Gemini service.

```
/**
 * Content-Based Recommendation Engine
 * Scoring weights:
```

```
 *   Location      : 30%
 *   Price         : 30%
 *   Bedrooms      : 20%
 *   Property Type : 10%
 *   Area          : 10%
 */
```

Listing 5.3: Weights of the Content-Based Recommendation Score

5.2 Test System Implementation

Testing checks that the system does what the requirements state and reveals defects before the system is used. Following Sommerville (2016), testing was organized in three stages: development testing, release testing and user testing.

5.2.1 Development Testing

Development testing is carried out by the developers while the system is being built. Table 5.3 shows the levels applied. The project contains automated test scripts for the authorization rules, the search and recommendation functions, the valuation module and the AI assistant, which are run against the local test environment.

Table 5.3: Levels of Development Testing

Level	What Was Tested
Unit testing	Individual functions, such as the recommendation score, the preparation of prediction features and the validation of forms
Component testing	Each Route Handler with valid and invalid input and with users of different roles
System testing	Complete workflows across the interface, services and database: property approval, rental and purchase

Evaluation of the price prediction model. The valuation module records the evaluation of the trained model. The model was trained on the 30 sample property records held in the development database, with 6 records kept aside as a test set, and was compared with three simple baseline estimates. Table 5.4 presents the recorded results. The model has a lower error than every baseline. These figures are indicative only: the sample is small and consists of sample listings with asking prices, not of collected Abudwak transaction data, so the results cannot be generalized to the real market. Ridge regression is the model used with the present sample, while the Random Forest and Gradient Boosting models are intended for use when a larger dataset is available.

Table 5.4: Recorded Evaluation of the Price Prediction Model

Method	MAE (USD)	RMSE (USD)	R^2
Baseline: overall median price	30,000	41,130	-0.25
Baseline: median by location and type	14,583	16,105	0.81
Baseline: price per square metre	45,536	54,348	-1.18

Method	MAE (USD)	RMSE (USD)	R^2
Ridge regression, test set (6 records)	6,424	7,638	0.96
Ridge regression, leave-one-out validation	8,289	15,394	0.77

5.2.2 Release Testing

Release testing checks a complete version of the system against its requirements before it is given to users. It was approached as requirements-based testing, in which each test case is derived from a functional requirement of Chapter Three. Table 5.5 presents the tests executed on the public portal in the local test environment, with the result observed for each.

Table 5.5: Release Tests Executed on the Public Portal

ID	Test Case	Expected and Observed Result	Status
RT-01	Open the home page	The home page is displayed with navigation and the property search button	Passed
RT-02	Open the properties page	Published listings are displayed with filters for city, listing type, property type, bedrooms and budget	Passed
RT-03	Open a property	The details page shows the specifications and status of the property	Passed
RT-04	Open price prediction	The form for city, type, bedrooms, bathrooms and area is displayed	Passed
RT-05	Open the AI assistant	The assistant panel opens with a welcome message and an input field	Passed
RT-06	Open the login page	The sign-in and create-account form is displayed	Passed

The role-based workflows are tested with one test account for each role. Table 5.6 lists the release test script for these workflows, with the result that the system is required to produce in each case.

Table 5.6: Release Test Script for the Role-Based Workflows

ID	Req.	Test Case	Expected Result
RT-07	FR-01	Log in as each role	Each user is redirected to the area of the user's role
RT-08	FR-01	Open another role's area	Access is refused and the user is returned to the user's own area
RT-09	FR-02, FR-03	Manager adds a property; Administrator approves it	The property becomes published and appears in the public listings
RT-10	FR-07	Customer opens recommendations	Properties are listed with a score and reasons
RT-11	FR-11	Customer submits a rental booking; Manager approves it	The rental becomes active and the property status becomes Rented
RT-12	FR-12	Manager creates a direct rental	The lease is created for the selected customer and property
RT-13	FR-13	Customer submits a purchase request; Administrator confirms; Customer pays; Administrator approves	The sale is completed and the property status becomes Sold

ID	Req.	Test Case	Expected Result
RT-14	FR-14	Open the receipt of a payment	An itemized receipt is displayed and can be printed
RT-15	FR-04	Search for a rented or sold property	The property is not offered as available

5.2.3 User Testing

User testing is the stage in which the intended users try the system in their own setting and decide whether it meets their needs. It is organized as acceptance testing with representatives of the three groups of users identified in Chapter Three: property seekers, property managers or agents, and an administrator. Each participant carries out the tasks of the participant's role from the test script in Table 5.6, for example searching for a property and submitting a rental booking, and the tester records whether each task was completed, the difficulties met and the comments of the participant. The acceptance criteria are that every task of the role can be completed without assistance from the developer and that the role boundaries are respected. The comments collected are used to correct defects and to improve the wording and layout of the interface.

5.3 Developing User Manuals

The user manual explains how each actor uses the system. It is organized by role.

Public Visitor. (1) Open the website in a web browser. (2) Select Search Properties, choose the city, listing type, property type, bedrooms and budget, and select Apply Filters. (3) Select View Details to see the specifications, amenities and location of a property. (4) To estimate a price, open the price prediction page, enter the city, property type, bedrooms, bathrooms and floor area, and submit the form. (5) To ask a question, select Ask AI Assistant and type the question. Figure 5.1 shows the property listing and the details page, and Figure 5.2 shows the AI assistant.

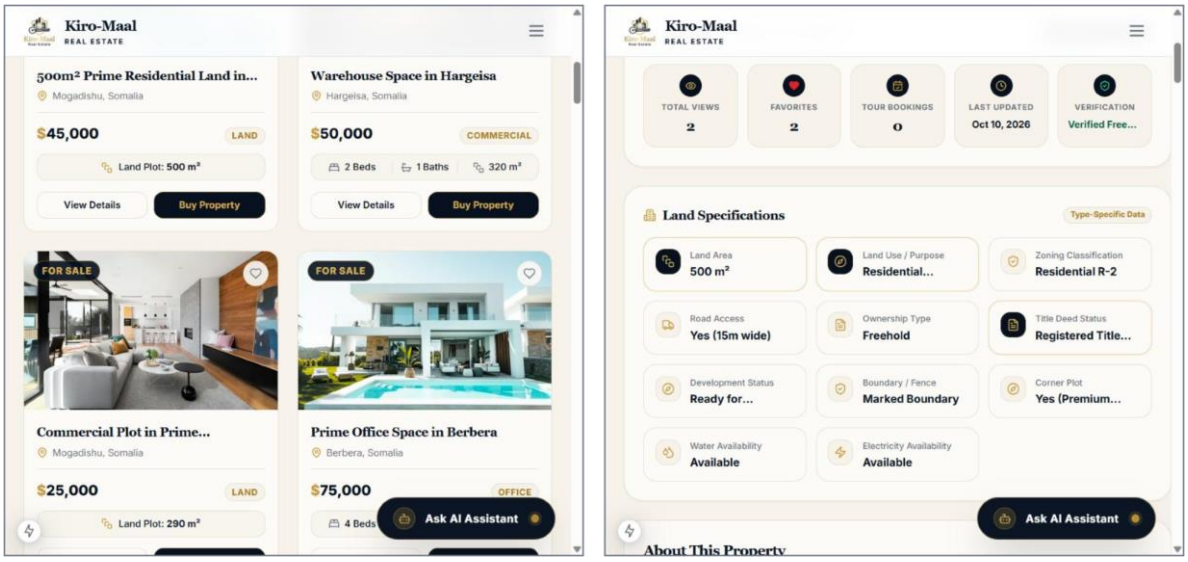


Figure 5.1: Property Listing and Property Details Page

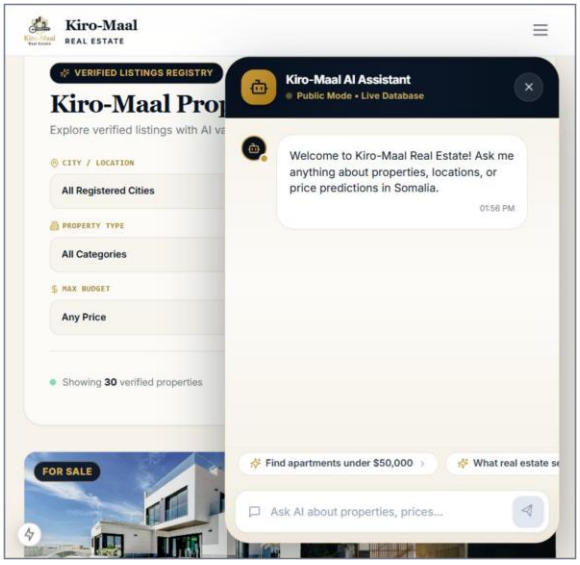


Figure 5.2: AI Assistant

Customer. (1) Select Create Account on the login page, enter the name, e-mail address and password, and sign in. (2) Select the heart icon on a property to save it as a favorite, and open Recommendations to see the properties suggested by the system. (3) To rent, open a property that is for rent, choose the rental period, complete the payment step and submit the booking; its status appears under My Rentals. (4) To buy, open a property that is for sale and submit a purchase request; when the Administrator has confirmed it, select Pay Now under My Purchases. (5) Open Transactions to view payments and download receipts.

Property Manager. (1) Sign in to open the manager dashboard. (2) Under Properties, select Add Property, complete the steps for details, pricing, images, location and amenities, and submit the property for approval. (3) Under Customers, register or search customers. (4) Under

Rentals, open a pending booking and approve or reject it, or create a rental directly by selecting the property and the customer and entering the period, rent and payment status. (5) Under Inquiries and Bookings, respond to inquiries and confirm, reschedule or cancel viewings.

Administrator. (1) Sign in to open the administrator dashboard. (2) Under Properties, open a pending property and approve it, reject it with a note, or publish it. (3) Under Requests, confirm a purchase request, and after the payment has been verified, approve the sale. (4) Under Users, create accounts, change roles, or suspend and reactivate accounts. (5) Use Transactions, Analytics and Audit Logs to monitor the activity of the platform, and Settings to configure cities and property categories.

5.4 Chapter Summary

This chapter described the implementation of the system. The coding phase presented the tools, the structure of the source code and examples of the database, access control and AI implementation. Testing was organized as development, release and user testing, the recorded evaluation of the price prediction model was reported with its limitations, and the release test cases were derived from the functional requirements. The chapter ended with the user manual for the four actors. The next chapter concludes the study.

CHAPTER SIX

CONCLUSION AND ENHANCEMENT

6.0 Introduction

This chapter concludes the study. It reviews the extent to which the objectives stated in Chapter One were achieved, discusses the weaknesses and problems of the system, and proposes future work.

6.1 Objective Achievement

The general objective of the project was to design and develop an AI-powered real estate recommendation and price prediction system that gives property seekers, sellers and investors a structured and data-driven alternative to the informal property market. This objective was met in the form of a working web-based system that combines a centralized property database, digital rental and sale workflows and three AI functions. Table 6.1 reviews the four specific objectives.

Table 6.1: Achievement of the Specific Objectives

Specific Objective	What Was Achieved	Status
To develop a hybrid AI recommendation engine using content-based and collaborative filtering	A recommendation engine that combines content-based scoring with preferences learned from each customer's searches, favorites and views. Filtering based on the behaviour of similar users requires more usage data	Partially achieved
To build a price prediction model using Random Forest and Gradient Boosting	A price prediction engine with ridge regression, Random Forest and Gradient Boosting models. It was trained and evaluated on a small sample of listings, not yet on collected Abudwak data	Partially achieved
To integrate a natural language interface for conversational interaction	An AI assistant that answers questions in natural language using current property data from the database	Achieved
To implement a centralized property database and management module	A normalized MySQL database with property management, approval, search, rental, sale, payment and reporting functions	Achieved

6.2 Weaknesses and Problems of the System

Limited data. The most important weakness is the data available to the AI functions. The price prediction model was trained on a small set of sample listings with asking prices, so its estimates are indicative and cannot yet be regarded as reliable valuations for Abudwak. For the same reason, recommendation based on the behaviour of similar users is limited until the platform has real users.

Payments. The system records and verifies payments by card, bank transfer, cash and mobile money, but it is not connected automatically to the payment providers. A payment made outside the system must be confirmed by the authorized role.

Dependence on external services and connectivity. The AI assistant depends on an external AI service and on an internet connection, and the system as a whole requires connectivity, which is not always stable in the target area.

Language and access. The interface is presented in English, which limits its use by property seekers who are more comfortable in Somali, and the system is available as a web application only.

Testing with users. The system was tested in a local test environment. Its behaviour with a large number of simultaneous users and its acceptance by a wide group of real users remain to be confirmed after deployment.

6.3 Future Work

The following enhancements are proposed. First, property and transaction data should be collected in Abudwak, in cooperation with local agents, and the price prediction models should be retrained and re-evaluated on these data, including the Random Forest and Gradient Boosting models. Second, collaborative filtering should be added to the recommendation engine when sufficient user activity has accumulated. Third, the system should be integrated with mobile money and card payment providers so that payments are confirmed automatically. Fourth, a Somali-language interface should be provided and the AI assistant should be extended to converse in Somali. Fifth, a mobile application and notifications by SMS would widen access for users with limited connectivity. Finally, the system should be deployed for public use and evaluated with real users over a period of time.

6.4 Chapter Summary

This chapter concluded the study of the AI Agent for Real Estate Recommendation and Price Prediction System. Two of the four specific objectives were achieved in full and two in part, the difference being due mainly to the limited data available. The weaknesses of the system were identified as limited data, manual confirmation of payments, dependence on external services, language and limited user testing, and future work was proposed to address each of them. The system nevertheless demonstrates that a structured, AI-assisted platform for the property market of Abudwak is feasible.